

Faculty of Sciences

Département of Computer Science



MASTER THESIS

Nauty (trouver un meilleur titre)

Auteur: Rodolphe Prévot

Promoteur : Robin Petit

Directeur : Gwenaël Joret

Academic year 2025–2026

Contents

1	Introduction	2
2	Preliminaries	4
2.1	Algebra Definitions	4
2.2	Graph Definitions	5
2.2.1	Definitions specifics to isomorphisms	5
2.3	Complexity Definitions	6
3	State of the Art	9
3.1	The Graph Isomorphism Problem	9
4	Nauty Algorithm	16
4.1	Introduction of the algorithm	16
4.2	Search Tree construction	16
4.2.1	Formal definitions of the search tree functions	18
4.2.2	Implementation strategies	21
4.2.3	Pruning strategies	26
4.2.4	Final Search Tree Construction	30

Chapter 1

Introduction

In graph theory, the graph isomorphism problem (GI) is a well-known challenge: determining whether two given graphs are isomorphic. Fully retracing its history is nearly impossible given the sheer number of works devoted to the topic, but we will review its most significant milestones. GI first appeared as a computational problem in the chemical documentation literature around the 1950s [49] (as cited in [26]), where it took the form of a matching problem between a “molecular graph” and graphs of the same type stored in a database. With this approach, we find ourselves at the origins of cheminformatics: the application of computer science to chemistry. Early instances of this matching problem gave rise to some of the first dedicated systems. Unger proposed GIT (**G**raph **I**somorphism **T**ester) [57], one of the earliest heuristics for testing the isomorphism of pairs of directed line graphs, while Sussenguth introduced a graph-theoretic algorithm specifically designed for matching chemical structures [55]. These early, domain-specific efforts laid the foundations for the more general algorithmic developments that followed.

Throughout the 20th century, considerable research efforts were devoted to solving GI, with varying degrees of efficiency. On the one hand, one of the simplest approaches is based on backtracking, a widely known and relatively easy-to-implement technique. However, it suffers from poor scalability and provides no guarantee of efficiency in the worst case [51]. On the other hand, more sophisticated methods were also proposed. One notable approach involved the construction of a so-called representative graph and a reordered graph to test for isomorphism [12]. Although this general approach was later shown to be incorrect (as cited in [20]), it played an important role in stimulating further investigations into the graph isomorphism problem.

Finally, a more promising direction emerged: reducing graphs to their canonical form. The idea is straightforward: every graph can be transformed into a *unique* canonical representation [2], meaning that two isomorphic graphs will share the same canonical form. This shift in perspective transformed GI from a problem of structural equivalence into one of equality: rather than directly checking whether two graphs are isomorphic, one simply compares their canonical forms. This change of viewpoint led to more efficient solutions.

In 1981, Brendan D. McKay introduced a breakthrough with his software *Nauty* (“**n**o **a**utomorphisms, **y**es?”) [39], which provided a much more efficient approach to tackling GI. Initially, the program had been started in 1976, written in Fortran and Assembly, and was originally called GLABC. However, in 1981, it was finalised in C and received

its new name: *Nauty*. Since then, the program has undergone continuous optimisation to maximise its speed and efficiency. The key idea behind *Nauty* is to compute a graph’s canonical form without relying on matrix representations. Instead, it uses a search tree combined with a partition refinement system to generate the canonical form.

Nowadays, this canonical form capability enables a powerful application: the exhaustive generation of non-isomorphic graphs. Given n vertices (and optionally additional constraints), the goal is to enumerate all possible graphs that are pairwise non-isomorphic. Isomorph-free exhaustive generation is a problem that researchers have been studying for decades. For instance, the generation of cubic graphs has been investigated since the 1960s, as these graphs are useful in various fields: in chemistry, they can model carbon networks; in mathematics and computer science, they often serve as minimal counterexamples in graph theory [9]. McKay, for his part, successfully connected his tool *Nauty* to the generation of non-isomorphic graphs [38].

Nauty is much more than just a tool for canonical labelling and the generation of non-isomorphic graphs. It has evolved into a comprehensive program supporting a wide range of graph families and offering additional features, such as graph statistics and other advanced analysis tools [42]. Furthermore, libraries based on *Nauty* have been developed for several programming languages: Pynauty in Python [17], NautyGraphs in Julia [18], and even pl-nauty in Prolog [21].

In practice, graph isomorphism tools are highly useful and widely used in scientific research. In general, these tools are employed to detect symmetries in combinatorial objects, which allows them to be applied across a variety of domains. In optimisation, for instance, GI tools are used in SAT solving, where symmetries can be exploited to reduce the search space [29]. Another application lies in data collections: for molecular databases, for example, it is sufficient to store the canonical form of a molecule rather than the molecule itself. Further applications can also be found in machine learning, software verification, and many other fields (as cited in [26]).

The main goal of this thesis is to re-implement one of the most efficient tools for graph isomorphism, *Nauty*, using a more modern programming language: C++. Recreating *Nauty* from scratch will allow us to evaluate optimisation differences, modernise its design, and extend its capabilities. For instance, the original *Nauty* does not support every possible graph family; in this updated version, we aim to include both some of the families already supported and additional ones, thereby contributing new features to the tool. The objective of this new codebase is certainly not to achieve better raw performance than the original *Nauty*, but rather to provide a modern perspective on the tool. Moreover, C++ remains a widely used programming language, and a C++ implementation of *Nauty* can make it more accessible to a broader audience while retaining better performance than more popular but interpreted languages such as Python.

This thesis is divided into four main chapters. The first introduces the fundamental concepts and preliminaries necessary to fully understand the key ideas discussed throughout this work. The second presents a comprehensive state of the art, summarising previous research on the graph isomorphism problem. The third is devoted to McKay’s canonical labelling algorithm and the construction of the automorphism group. Finally, the fourth chapter presents our C++ implementation of *Nauty* and discusses the results obtained.

Chapter 2

Preliminaries

This chapter introduces all the definitions necessary to understand this document. For further details on the following concepts, refer to Reinhard Diestel's book on graph theory [16].

2.1 Algebra Definitions

Definition 1 (informal). An *isomorphism* is a one-to-one correspondence (mapping) between two sets that preserves binary relationships between elements of the sets. If an element of one set is mapped to an element of the other set, this is denoted by $x \mapsto y$, meaning that the element x is mapped to the element y .

An *automorphism* is an isomorphism from a set to itself.

Definition 2. A *group* is a set with a binary operation that satisfies the following constraints: the operation is associative, it has an identity element, and every element of the set has an inverse. For example, $(\mathbb{R}, +)$ is a group: it consists of the set of real numbers with addition as the binary operation. It has an identity element: 0, every element has an inverse element (each number with its opposite) and it's associative: for all numbers a, b, c in $\mathbb{R}$: $a + (b + c) = (a + b) + c$.

A *homomorphism* is a transformation of one set into another that preserves in the second set the relations between elements of the first. For example, given two groups, $(G, \cdot)$ and $(H, \circ)$, a *group homomorphism* from $(G, \cdot)$ to $(H, \circ)$ is a function $\phi : G \rightarrow H$ such that $\phi(g_1 \cdot g_2) = \phi(g_1) \circ \phi(g_2)$ for every $g_1, g_2 \in G$.

Let $(G, *)$ be a group with identity element e , and let X be a non-empty set. A *group action* of G on X exists if there is a map

$$\cdot : G \times X \rightarrow X, \quad (g, x) \mapsto g \cdot x$$

such that the following two properties hold:

1. Identity: $\forall x \in X, \quad e \cdot x = x$,
2. Compatibility: $\forall g, h \in G, \forall x \in X, \quad (g * h) \cdot x = g \cdot (h \cdot x)$.

As an example, let S_n denote the *symmetric group* acting on a set X . We indicate the action of elements of S_n by exponentiation: for $x \in X$ and $g \in S_n$, x^g denotes the image of

x under g . The same notation is used to indicate the induced action on complex structures derived from X . As additional notation, for $W \subseteq V$, we write $W^g = \{w^g : w \in W\}$.

Definition 3. Let a group G act on a set X , and let $S \subseteq X$. The *pointwise stabilizer* of S in G is the subgroup defined by:

$$G_{(S)} = \{g \in G \mid \forall x \in S, g \cdot x = x\}.$$

Definition 4. Let $G \leq S_n$ be a permutation group and let $\alpha \in \{1, \dots, n\}$. The *orbit* of α under the action of G is the set

$$\text{Orb}_G(\alpha) = \{\alpha^g \mid g \in G\}.$$

Definition 5. Let $G \leq S_n$ be a permutation group and let $\alpha \in \{1, \dots, n\}$. The *stabilizer* of α in G is the subgroup

$$G_\alpha = \{g \in G \mid \alpha^g = \alpha\}.$$

2.2 Graph Definitions

Definition 6. A *graph* is a pair $G = (V, E)$ of sets satisfying $E \subseteq \binom{V}{2}$; thus, the elements of E are 2-element subsets of V . The elements of V are the *vertices* of the graph G , the elements of E are its *edges*.

Let V^* denote the set of finite sequences of vertices. For $\sigma \in V^*$, $|\sigma|$ denotes the number of components of σ . If $\sigma = (v_1, v_2, \dots, v_k) \in V^*$ and $w \in V$ then $\sigma \parallel w$ denotes $(v_1, v_2, \dots, v_k, w)$. Furthermore, for $0 \leq s \leq k$, $[\sigma]_s := (v_1, v_2, \dots, v_s)$.

An acyclic graph (one not containing any cycles) is called a *forest*. A connected forest is called a *tree*. Each vertex of a tree is called a *node*, and a node of degree 1 (only connected to one node) is called a *leaf*. When a particular node is designated as the starting point, that node is called the *root*. Given a tree T , a *subtree* T' is a connected subgraph of T that is itself a tree. We write $T' \subseteq T$ to denote that T' is a subtree of T .

Let $G = (V, E)$ be a graph. Let $\sim$ be an equivalence relation on V . The *quotient graph* of G with respect to $\sim$, denoted $G/\sim$, is a graph (V', E') such that:

1. $V' = V/\sim = \{[v] \mid v \in V\}$
2. $E' = \{[u][v] \mid uv \in E\}$

Definition 7. We let $\mathcal{G}_n$ be the set of all graphs on n vertices (without loss of generality, say $V = \{1, 2, \dots, n\}$). $\mathcal{G}$ represents the set of all graphs.

Definition 8. The *star graph* $K_{1,3}$ is a tree with one internal node and three leaves. It is also known as the *claw graph*. The internal node is called the *center* of the star graph. Pour la majorité de nos exemples, nous allons utiliser ce graphe.

2.2.1 Definitions specifics to isomorphisms

Definition 9. A *colouring* of V (or of $G \in \mathcal{G}$) is a surjective function π from V onto $\{1, 2, \dots, k\}$ for some k . The number of colours, i.e. k , is denoted by $|\pi|$. A cell of π is

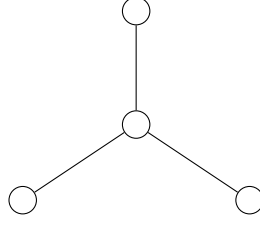


Figure 2.1: The star graph $K_{1,3}$ with its center and leaves.

the set of vertices with some given colour; that is, $\pi^{-1}(j)$ for some j with $1 \leq j \leq |\pi|$. We let Π be the set of all colourings.

If π, π' are colourings, then π' is *finer than or equal* to π , written $\pi' \preceq \pi$, if $\pi(v) < \pi(w) \Rightarrow \pi'(v) < \pi'(w)$ for all $v, w \in V$ (This implies that each cell of π' is a subset of a cell of π).

Here, the colouring is not related to a *proper colouring*, π is related to a *partitioning*.

A *discrete colouring* is a colouring in which each cell is a singleton, in which case $|\pi| = n$.

A *coloured graph* is a pair (G, π) where π is a colouring of G .

A colouring of a graph is called *equitable* if any two vertices of the same colour are adjacent to the same number of vertices of each colour. This is also called an *equitable/stable partition*.

Definition 10. Let S_n denote the *symmetric group* acting on V .

There are several notations to consider:

1. if π is a colouring of V , then π^g is the colouring with $\pi^g(v^g) = \pi(v)$ for each $v \in V$.
2. if (G, π) is a coloured graph, then $(G, \pi)^g = (G^g, \pi^g)$.

These properties show that two coloured graphs (G, π) and (G', π') are isomorphic iff there exists $g \in S_n$ such that $(G', \pi') = (G, \pi)^g$. In this case, we write $(G, \pi) \cong (G', \pi')$.

$\text{Aut}(G, \pi) = \{g \in S_n : (G, \pi)^g = (G, \pi)\}$; $\text{Aut}(G, \pi)$ is the *Automorphism group* of coloured graph (G, π) .

Definition 11. The *canonical form* is a function

$$C : \mathcal{G} \times \Pi \rightarrow \mathcal{G} \times \Pi$$

such that for all $G \in \mathcal{G}, \pi \in \Pi$ and $g \in S_n$:

1. $C(G, \pi) \cong (G, \pi)$.
2. $C(G^g, \pi^g) = C(G, \pi)$.

2.3 Complexity Definitions

For more informations on the complexity definitions, you can refer to Michael Sipser's book: Introduction to the Theory of Computation [54]

Definition 12 (minimalist). The class **NP** (comes from nondeterministic polynomial) is the set of decision problems for which the problem instances have certificates verifiable in polynomial time by a deterministic Turing machine, or alternatively the set of problems that can be solved in polynomial time by a nondeterministic Turing machine.

A decision problem is a member of **coNP** iff its complement is in the complexity class **NP**.

A decision problem B is **NP-complete** if it satisfies the two following conditions:

1. B must belong to the class **NP**.
2. Every problem in **NP** can be reduced to B in polynomial time.

Definition 13. A problem is **NP-intermediate** (**NPI**) if it is not **NP-complete** nor **P**.

Definition 14. A language L belongs to the class **SPP** (**Stoic PP**) if there exists a polynomial-time nondeterministic Turing machine M such that

$$x \in L \iff \#acc_M(x) = 1 \text{ and } x \notin L \iff \#acc_M(x) = 0,$$

where $\#acc_M(x)$ denotes the number of accepting computation paths of M on input x .

Definition 15. **LOGSPACE** (or **L**) is the complexity class of decision problems that can be solved by a deterministic Turing machine using a logarithmic amount of writable memory.

Definition 16. An *oracle* for a language B is an external device that is capable of reporting whether any string w is a member of B . An oracle Turing machine is a modified Turing machine that has the additional capability of querying an oracle. We write M^B to describe an oracle Turing machine that has an oracle for language B .

The *Polynomial-Time Hierarchy* (**PH**) is a hierarchy of complexity classes that generalizes the classes **NP** and **coNP**. Formally, the base level of the hierarchy is defined as:

$$\Delta_0^P := \Sigma_0^P := \Pi_0^P := P,$$

where **P** denotes the class of decision problems solvable in polynomial time by a deterministic Turing machine. Then, for every $i \geq 0$, the hierarchy is defined recursively as follows:

- $\Delta_{i+1}^P := P^{\Sigma_i^P}$
- $\Sigma_{i+1}^P := NP^{\Sigma_i^P}$
- $\Pi_{i+1}^P := coNP^{\Sigma_i^P}$

Here, P^A denotes the class of problems solvable in polynomial time by a deterministic Turing machine equipped with an oracle for some complete problem in class A . The classes NP^A and $coNP^A$ are defined analogously, using nondeterministic and co-nondeterministic Turing machines, respectively.

For example, $\Sigma_i^P = NP$, $\Pi_i^P = coNP$, and $\Delta_2^P := P^{NP}$ is the class of problems solvable in polynomial time with access to an oracle for an **NP-complete** problem.

This definition implies that each level of the hierarchy is contained within the next. That is:

- $\Sigma_i^P \subseteq \Delta_{i+1}^P \subseteq \Sigma_{i+1}^P$

- $\Pi_i^P \subseteq \Delta_{i+1}^P \subseteq \Pi_{i+1}^P$

The union of all classes in this hierarchy forms the polynomial-time hierarchy, denoted by PH.

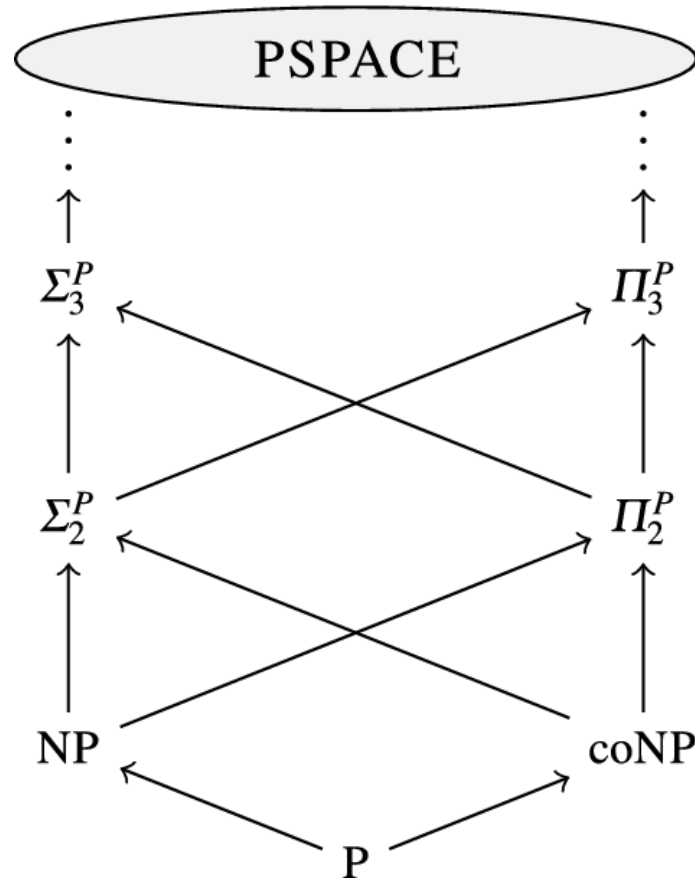


Figure 2.2: Polynomial Time Hierarchy where the arrows denote the inclusion: image from [6]

Chapter 3

State of the Art

3.1 The Graph Isomorphism Problem

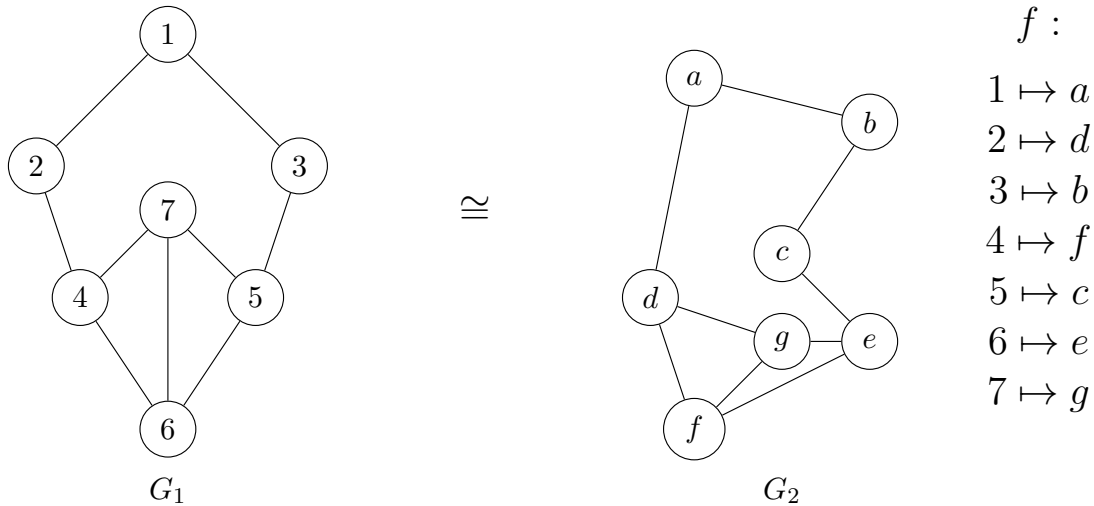


Figure 3.1: Example of isomorphism between two graphs G_1 and G_2

The Graph Isomorphism (GI) problem is one of the best-known problems in theoretical computer science and graph theory. It has attracted so much attention from algorithm designers that it was referred to as a “disease” by researchers [50]. GI belongs to the complexity class NP (cf. Definition 12), yet it remains unknown whether it can be solved in polynomial time. In other words, it is still open whether GI belongs to P. Likewise, GI is not known to be NP-complete, and its relationship with coNP also remains unresolved. As a result, the Graph Isomorphism problem continues to occupy a unique position in computational complexity theory.

During the 1970s, GI became one of the emblematic “open problems” of complexity theory: it was unknown whether it belonged to P, was NP-complete, or occupied an intermediate position between the two [22]. Today, the possibility that GI is NP-complete is considered highly unlikely. Indeed, it has been shown that proving GI NP-complete would imply that the polynomial-time hierarchy (PH) collapses to its second level [23, 52]. Such a collapse is considered extremely unlikely, as complexity theorists widely conjecture that all levels of

the polynomial hierarchy are distinct. Consequently, GI is regarded as one of the strongest candidates for an NP-intermediate problem.

Although its exact complexity remains unknown, several important theoretical results have progressively clarified GI’s position within the complexity landscape. For instance, Graph Isomorphism has been shown to belong to **SPP**. Furthermore, Torán proved that GI is hard for a large collection of logarithmic-space complexity classes, highlighting that the problem also occupies a central role in the study of space complexity [56]. These results illustrate that, despite decades of research, the exact complexity of GI remains remarkably subtle.

Progress has also been made from an algorithmic perspective. For more than thirty years, the fastest proven algorithm for Graph Isomorphism had a running time of $e^{\mathcal{O}(\sqrt{n \log n})}$ [5]. A major breakthrough came in 2016, when Babai introduced the first quasi-polynomial-time algorithm, running in time $e^{\log(n)^{\mathcal{O}(1)}}$ [3]. The algorithm builds upon previous advances in Graph Isomorphism algorithms and handles highly symmetric cases using group-theoretic (introduced in more details in a few paragraphs) local certificates¹ and combinatorial canonical partitioning techniques. Although this result does not settle whether GI belongs to P, it further supports the belief that the problem is NP-intermediate rather than NP-complete, since an NP-complete problem admitting a quasi-polynomial-time algorithm would imply that every problem in NP also has a quasi-polynomial-time algorithm.

Despite the difficulty of the general Graph Isomorphism problem, efficient algorithms can be obtained for specific classes of graphs. In particular, Babai, Erdős, and Selkow showed that in the Erdős–Rényi random graph model $\mathcal{G}(n, 1/2)$, where each edge is independently included with probability 1/2, almost all graphs can be canonically labeled in linear time by a simple refinement procedure [4]. This result illustrates that, although carefully constructed graphs can remain challenging, random graphs are typically easy to distinguish due to their lack of structural symmetries.

Despite these advances, the precise complexity of Graph Isomorphism remains unresolved. Nevertheless, polynomial-time algorithms are known for numerous restricted graph classes. These classes are typically characterised by structural constraints, such as excluded minors² (see [48] in Russian, as cited in [40]), or excluded topological subgraphs³ [24]. Prominent examples include graphs of bounded degree⁴ [36], bounded tree-width [8]⁵, and, more recently, graphs of clique-width⁶ at most three [14]. Graphs of bounded genus⁷ were also claimed to admit a polynomial-time algorithm [19]; however, the correctness of the original proof was later questioned [44].

For some restricted graph classes, even stronger complexity bounds are known. In particular, Graph Isomorphism for planar graphs⁸ and interval graphs⁹ can be solved in

¹efficiently verifiable structures that reveal properties of local graph structures without requiring a complete global analysis

²A graph H that cannot be formed within another graph G through any sequence of edge deletions, vertex deletions or edge contractions

³Same idea of a minor but only allow for the contraction of edges adjacent to degree-two

⁴A graph whose maximum degree is bounded by a fixed constant d .

⁵An integer number which specifies, informally, how far the graph is from being a tree

⁶It measures the structural complexity of a graph by the minimum number of labels required to construct it using simple graph operation

⁷The genus of a graph is the minimum number of handles required on a surface to embed the graph without edge crossings

⁸graph that can be drawn on a flat surface so that its edges intersect only at their shared endpoints

⁹An undirected graph formed from a set of intervals on the real line.

LOGSPACE (cf. [Definition 15](#)), representing a significant improvement in memory complexity [15, 34].

As increasingly sophisticated techniques have been developed, polynomial-time algorithms have been obtained for a growing number of graph classes. However, many of these algorithms remain primarily of theoretical interest and are difficult to implement efficiently. Notable exceptions include [trees](#) [1], planar graphs [27], and interval graphs [11], for which practical algorithms are available.

Conversely, restricting the input to particular graph families does not necessarily simplify the problem. Indeed, Graph Isomorphism remains GI-complete on several seemingly restricted classes, including bipartite graphs¹⁰, chordal graphs¹¹, graphs of bounded degeneracy¹², and graphs of bounded expansion¹³ [24].

To solve the Graph Isomorphism (GI) problem, three main types of algorithms can be considered [24]: graph-theoretic algorithms, group-theoretic algorithms, and combinatorial algorithms.

The first family consists of *graph-theoretic algorithms*, classical algorithms that exploit structural properties of specific graph classes. A representative example is the algorithm of Hopcroft and Wong [28] for planar graphs, which relies on the decomposition of the graph into triconnected (3-connected) components and uses graph-theoretic techniques to solve the isomorphism problem efficiently.

The second family comprises *group-theoretic algorithms*, based on computing generators for the automorphism group of a graph. These methods leverage group-theoretic properties and have dominated GI research since the early 1980s. Indeed, GI and the problem of computing a generating set for a graph’s automorphism group can each be reduced to the other in polynomial time [37]: the two problems are polynomial-time equivalent, so that solving either one suffices to solve the other. This is the case of Luks’ algorithm [36], which solves the isomorphism problem on graphs of bounded degree by exploiting the structure of the automorphism group to reduce the problem to a sequence of simpler instances.

Finally, *combinatorial algorithms* are simple, general-purpose procedures that do not rely on the specific structure of graph classes. All combinatorial algorithms are based on the idea of *colour refinement* (the 1-dimensional version of the Weisfeiler-Leman algorithm [31]), which aims to compute a *stable colouring* of a graph, as illustrated in [Algorithm 1](#). Colour refinement is one of the simplest ideas for testing graph isomorphism and has been refined, extended, and incorporated into numerous algorithms over the years. However, the earliest version of the algorithm dates back to 1965, when it was introduced by Morgan for the study of chemical structures [43].

To use colour refinement as an isomorphism test, we apply it to the disjoint union of the input graphs G and G' . The algorithm *distinguishes* G and G' if there exists a colour c in the stable colouring such that G and G' contain different numbers of vertices of colour c . Colour refinement *identifies* a graph G if it distinguishes G from all non-isomorphic

¹⁰a network of vertices divided into two disjoint sets, where every edge connects a vertex in one set to a vertex in the other

¹¹An undirected graph where every cycle of length 4 or greater has a chord

¹²graph has bounded degeneracy d if every subgraph contains a vertex with a degree of d or less

¹³Graphs of bounded expansion are families of sparse graphs (a graph whose number of edges is small compared to its number of vertices) where all shallow minors (a minor obtained by restricting the depth of the contractions) have bounded average density

Algorithm 1 Colour Refinement Algorithm [24]

```
1: Initialisation: Assign the same initial colour to all vertices.
2: repeat
3:   Refinement Step:
4:   for all pairs of vertices  $v, w \in V$  do
5:     for all colours  $c$  in the current colouring do
6:       if the number of neighbours of  $v$  of colour  $c$  differs from that of  $w$  then
7:         Assign different colours to  $v$  and  $w$ 
8:       end if
9:     end for
10:  end for
11: until (the colouring is stable)
```

graphs; that is, no non-isomorphic graph shares the same stable colouring as G . However, if two graphs share the same stable partition, this is not a sufficient condition to prove that they are isomorphic; it is only a necessary one. Therefore, colour refinement does not fully solve the Graph Isomorphism problem, but rather provides a useful indication.

Colour refinement is not only used in the context of Graph Isomorphism; it also plays a role in reducing the cost of solving linear programs [25], as well as in computing graph kernels [53].

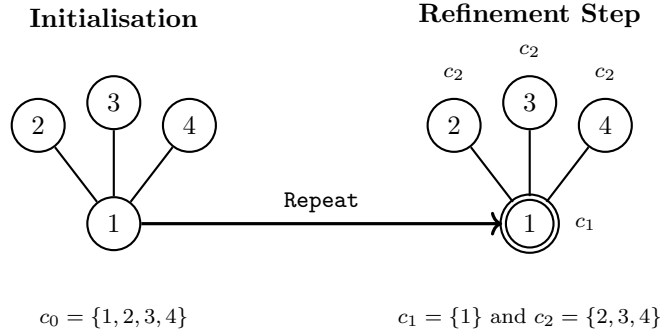


Figure 3.2: Example of colour refinement on $K_{1,3}$.

Regarding the time complexity, for a graph G with n vertices and m edges, a stable colouring can be reached after at most n refinement steps. In fact, it can be computed in time $\mathcal{O}((n+m) \log n)$ [10], and has a lower bound of $\Omega((n+m) \log n)$ for the natural class of iterative partition refinement algorithms [7]. This gives a tight bound of $\Theta((n+m) \log n)$ for such algorithms.

Now that the concept of colour refinement is well defined, we can turn to the types of combinatorial algorithms that build upon it. The most direct generalisation is the *Weisfeiler-Leman algorithm* (or k -WL) [31]. Its underlying principle remains largely the same as that of colour refinement, except that instead of comparing individual vertices, the algorithm operates on k -tuples of vertices. The central concept it introduces is that of i -neighbours: for $\vec{v}, \vec{w} \in V^k$ and $i \in \{1, \dots, k\}$, the tuples $\vec{v}$ and $\vec{w}$ are said to be i -neighbours if $v_j = w_j$ for all $j \neq i$.

An interesting result is that for all graphs G and H , 2-WL can distinguish G and H if and

Algorithm 2 k -dimensional Weisfeiler-Leman (k -WL) algorithm

Require: Graph $G = (V, E)$, integer $k \geq 1$

Ensure: Stable colouring of V^k

```
1: Initialization:
2: for all tuples  $\vec{v}, \vec{w} \in V^k$  do
3:   if  $\vec{v} \not\cong \vec{w}$  then                                 $\triangleright$  No isomorphism between induced subgraphs
4:     Assign different initial colours to  $\vec{v}$  and  $\vec{w}$ 
5:   end if
6: end for
7: repeat
8:   Refinement step:
9:   for all tuples  $\vec{v}, \vec{w} \in V^k$  do
10:    for  $i = 1$  to  $k$  do
11:      Compare multisets of colours of  $i$ -neighbours of  $\vec{v}$  and  $\vec{w}$ 
12:      if they differ then
13:        Assign different colours to  $\vec{v}$  and  $\vec{w}$ 
14:      end if
15:    end for
16:  end for
17: until the colouring is stable
```

only if colour refinement can distinguish them. In other words, despite the fact that 2-WL operates on pairs of vertices and provides a richer representation of the graph structure, it has the same distinguishing power as colour refinement for the graph isomorphism problem [24].

However, the situation changes when considering dimensions higher than two. For every $k \geq 3$, k -WL is strictly more powerful than $(k - 1)$ -WL. In particular, while 2-WL and colour refinement have the same distinguishing power, 3-WL can distinguish graph pairs that are indistinguishable by 1-WL and 2-WL.

The reason behind this improvement is that k -WL considers tuples of k vertices instead of individual vertices. Consequently, it can capture more global structural relationships between vertices. In contrast, 1-WL has a more local perspective, as it only refines vertex colours based on the multiset of colours of their neighbours. By increasing k , the algorithm is able to analyse interactions between several vertices simultaneously and therefore detect structural differences that remain invisible to lower-dimensional versions.

For example, 3-WL can solve the graph isomorphism problem for planar graphs [32]. This result illustrates the increasing expressive power of higher-dimensional Weisfeiler-Leman algorithms.

Regarding its complexity, for a graph G with n vertices, the stable colouring of k -WL can be reached after at most n^k refinement rounds. The stable colouring can then be computed in time $\mathcal{O}(n^k \log n)$.

Beyond this tuple-based generalisation, applying higher-dimensional Weisfeiler-Leman algorithms quickly becomes computationally expensive in practice. Consequently, most modern graph isomorphism software relies on the one-dimensional version, namely *colour refinement*, introduced in the previous section. For the vast majority of graphs, colour refinement already provides a sufficiently informative partition of the vertex set.

When colour refinement alone fails to distinguish two graphs, modern isomorphism algorithms typically resort to a second family of techniques known as *individualisation-refinement*. This approach combines the individualisation of carefully chosen vertices with repeated applications of colour refinement. The technique was first introduced by Parris and Read [46], as cited in [40]. Individualising a vertex breaks graph symmetries, allowing colour refinement to be applied again and often producing a significantly finer partition of the vertices [26].

The first practical implementation of this paradigm appeared in *Nauty*, whose central idea is to construct a *search tree* in which individualisation and refinement are applied recursively. During the exploration of this search tree, a *node invariant* is computed at every node to guide the search and prune unnecessary branches. Once the traversal is complete, the information gathered throughout the search is used to compute a *canonical labelling* of the graph, that is, a unique representation (or canonical form) of the graph. This canonical labelling lies at the heart of solving the Graph Isomorphism problem, since two graphs are isomorphic if and only if their canonical forms are identical. These notions will be revisited in much greater detail in the next chapter.

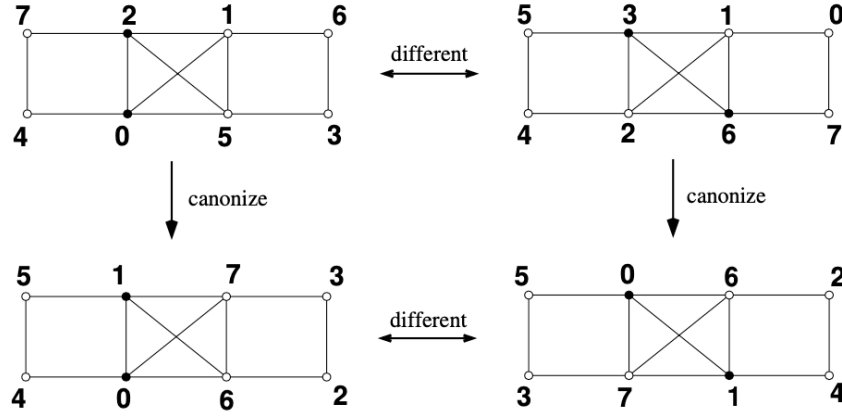


Figure 3.3: Example of GI with canonical labelling from [42]

Over time, several other GI solvers based on the individualisation-refinement paradigm have emerged, each building on or extending the ideas underlying *Nauty*. *Saucy*, introduced in 2004, follows the same principle but was designed to outperform *Nauty* on sparse graphs [13]. A few years later, *Bliss* was released; it also computes canonical forms but incorporates additional techniques that improve performance on more complex graphs ([30], as cited in [40]). *Conauto*, introduced in 2009, has shown remarkable efficiency on random graphs and on certain families of hard instances [35]. Finally, *Traces* was developed [47] as a tool able to compete with *Nauty* on the Graph Isomorphism problem; today, *Traces* and *Nauty* are tightly integrated and distributed together [40].

Unfortunately, despite their remarkable practical performance, all the individualisation-refinement algorithms cannot achieve polynomial worst-case running time. In 2017, David Neuen and Pascal Schweitzer proved that every algorithm belonging to this family has an exponential lower bound [45]. In other words, there exist graph families on which every individualisation-refinement algorithm necessarily requires exponential time. Nevertheless, these algorithms remain the most successful practical approach for solving the Graph Isomorphism problem. This striking contrast illustrates how far practical algorithms still

are from the best theoretical results, such as Babai's quasi-polynomial-time algorithm.

Finally, a rather different approach to the Graph Isomorphism problem is based on linear programming. Let A and B denote the adjacency matrices of two graphs G and H . Then, G and H are isomorphic if and only if there exists a permutation matrix X such that

$$X^\top AX = B.$$

This formulation can be relaxed using LP techniques, notably the Sherali-Adams hierarchy. In particular, level- k Sherali-Adams relaxations offer a powerful framework that has been investigated for graph isomorphism testing [24].

Chapter 4

Nauty Algorithm

In this section, we will focus extensively on McKay’s general algorithm: Nauty.

4.1 Introduction of the algorithm

Nauty relies on the notion of a **canonical form** to achieve its main objective: finding a canonical labeling of a graph and, consequently, determining its automorphism group. The canonical form plays a central role in practice, as it constitutes a powerful tool for computing a canonical labeling.

The canonical form allows us to bypass the intrinsic difficulty of the graph isomorphism problem. Instead of directly comparing two graphs, we associate each graph with a unique and standardized representative, called its canonical graph (or canonical form). Two graphs are then isomorphic if and only if their canonical forms are identical. In this way, an equivalence problem is transformed into an equality problem, which is significantly easier to handle computationally.

We begin by introducing the algorithm itself and describing the different functions that compose it, as well as the process used to compute the canonical form of a graph. We then explain how the automorphism group can be constructed from this canonical form.

4.2 Search Tree construction

First of all, the algorithm itself is built upon a very specific type of **tree**: a search tree.

Informally, a search tree is a tree structure in which each node represents a state, and each branch corresponds to a specific action leading to another state. Since different actions produce different branches, the tree allows us to systematically explore all possible states starting from an initial state, with the goal of reaching a desired final state. In other words, we are performing a search over a tree structure, which explains the term *search tree*.

However, its particularity is that the leaves of this tree correspond to sequences of vertices **of a coloured graph**. As we move down the tree, these sequences grow longer. These sequences define a colouring of the graph obtained by first assigning unique colours to the vertices in the sequence and then deducing, in a “controlled” manner, the colouring

of the remaining vertices. The leaves of this tree correspond to sequences whose derived colouring is discrete. The root of this tree is therefore the “empty” sequence.

To correctly construct this tree and find our canonical form, we will need several functions:

- *A refinement function*: Starting from a partition, this function allows us to construct a new partition that is finer than the previous one.
- *A target cell selection function*: Starting from a partition, this function allows us to choose a cell from this partition in order to create the “child” of the previous partition.
- *A node invariant function*: A function that allows us to efficiently compare the different nodes/leaves of our search tree in order to determine the canonical form of the graph.

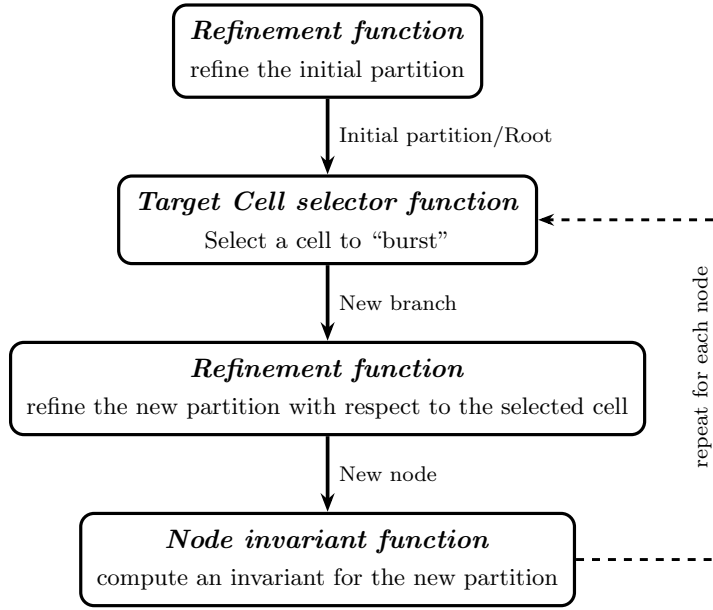


Figure 4.1: *search tree* construction in *Nauty*

These three functions are the fundamental components required to construct our search tree and find our canonical form. Before defining them formally, we will informally explain how they work and how they interact with each other to build our search tree.

Starting from a graph, we first apply the refinement function to obtain a so-called equitable partition (we will come back to this later when discussing the implementation of this function). This partition constitutes the root of our search tree.

Then, as long as we do not have a discrete partition, we apply the target cell selector to choose a cell from the current partition. Each branch represents a choice of element from the parent node’s selected cell. This cell is then split into two cells using an individualisation method (which will also be detailed later). For each branch, we apply the refinement function again to obtain a new partition, thereby creating a new node. Finally, on each node, we apply the node invariant function, which will be useful for determining the canonical form of our graph. We repeat this process until we obtain discrete partitions, which correspond to the leaves of our search tree. In summary, each node represents a partition, and each branch represents a cell chosen from the parent node to be individualised.

However, there exist additional functions that are more specific to the implementation of the algorithm and that will be explained in the implementation section of this document.

4.2.1 Formal definitions of the search tree functions

Now that we have introduced the different functions composing our algorithm, we can formally define them and establish the properties of the associated search tree.

Definition 17. The *Refinement function* is a function

$$R : \mathcal{G} \times \Pi \times V^* \rightarrow \Pi$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$,

R1. $R(G, \pi, \sigma) \preceq \pi$.

R2. if $\sigma_i \in \sigma$, then $\{\sigma_i\}$ is a cell of $R(G, \pi, \sigma)$.

R3. for any $g \in S_n$, we have $R(G^g, \pi^g, \sigma^g) = R(G, \pi, \sigma)^g$.

In other words, the refinement function returns a colouring that is finer than π , and any vertex belonging to the sequence σ must form a singleton cell in the refined colouring. Furthermore, the function is equivariant under permutation: applying any $g \in S_n$ to the graph, the colouring, and the vertex sequence before or after the refinement yields the same result.

Definition 18. The *Target cell selector function* is a function

$$T : \mathcal{G} \times \Pi \times V^* \rightarrow 2^V$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$

T1. if $R(G, \pi_0, \sigma)$ is discrete, then $T(G, \pi_0, \sigma) = \emptyset$.

T2. if $R(G, \pi_0, \sigma)$ is not discrete, then $T(G, \pi_0, \sigma)$ is a non-singleton cell of $R(G, \pi_0, \sigma)$.

T3. for any $g \in S_n$, we have $T(G^g, \pi^g, \sigma^g) = T(G, \pi, \sigma)^g$.

In other words, when the refinement function returns a discrete partition (corresponding to a leaf of the search tree) the target cell selector returns $\emptyset$. When the partition is non-discrete (corresponding to an internal node) it returns a non-singleton cell of that partition. As with the refinement function, the target cell selector is equivariant under permutation: applying any $g \in S_n$ to the inputs before or after the selection yields the same result.

Now that all the formal definitions of the tree have been established, we can define the search tree $\mathcal{T}(G, \pi_0)$, which depends on an initial coloured graph (G, π_0) . All the nodes of the tree are elements of V^* . This search tree has some characteristics to keep in mind:

- The root of $\mathcal{T}(G, \pi_0)$ is the empty sequence $()$.
- if σ is a node of $\mathcal{T}(G, \pi_0)$, let $W = T(G, \pi_0, \sigma)$. Then the children of σ are $\{\sigma \parallel w : w \in W\}$.
- a node σ of $\mathcal{T}(G, \pi_0)$ is a leaf iff $R(G, \pi_0, \sigma)$ is discrete.
- for any node σ of $\mathcal{T}(G, \pi_0)$, define $\mathcal{T}(G, \pi_0, \sigma)$ to be the subtree of $\mathcal{T}(G, \pi_0)$ consisting of σ and all its descendants.

To demonstrate the correctness of the algorithm, the pruning procedure that we will introduce, and to prove that we indeed obtain the canonical form of the graph, we need to establish several properties of this search tree.

Lemma 1. *For any $G \in \mathcal{G}$, $\pi_0 \in \Pi$, $g \in S_n$ we have $\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$.*

Proof. Let $\sigma = (\sigma_1, \dots, \sigma_k)$ be a node of $\mathcal{T}(G, \pi_0)$. We first show that $\mathcal{T}(G, \pi_0)^g \subseteq \mathcal{T}(G^g, \pi_0^g)$. Such that $\sigma^g = (\sigma_1^g, \dots, \sigma_k^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

We proceed by induction on s , for $0 \leq s \leq k$, showing that $[\sigma^g]_s = (\sigma_1^g, \dots, \sigma_s^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

Base case ($s = 0$): The empty sequence $()$ is the root of any search tree. So $[\sigma^g]_0$ is trivially a node of $\mathcal{T}(G^g, \pi_0^g)$.

Inductive step: Assume $[\sigma^g]_s = (\sigma_1^g, \dots, \sigma_s^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

Since $(\sigma_1, \dots, \sigma_s, \sigma_{s+1})$ is a node of $\mathcal{T}(G, \pi_0)$, the element σ_{s+1} was individualised to go from level s to level $s + 1$. Since we apply g to the entire structure, σ_{s+1} is also mapped to σ_{s+1}^g , which is equivalent to directly individualising σ_{s+1}^g in G^g with π_0^g . Hence $[\sigma^g]_{s+1} = (\sigma_1^g, \dots, \sigma_s^g, \sigma_{s+1}^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

By induction, σ^g is a node of $\mathcal{T}(G^g, \pi_0^g)$, hence $\mathcal{T}(G, \pi_0)^g \subseteq \mathcal{T}(G^g, \pi_0^g)$.

The reverse inclusion follows by applying the same argument with g^{-1} in place of g , which gives $\mathcal{T}(G^g, \pi_0^g)^{g^{-1}} \subseteq \mathcal{T}(G, \pi_0)$, and therefore $\mathcal{T}(G^g, \pi_0^g) \subseteq \mathcal{T}(G, \pi_0)^g$. $\square$

Corollary 2. *Let σ be a node of $\mathcal{T}(G, \pi_0)$ and let $g \in \text{Aut}(G, \pi_0)$. Then σ^g , is a node of $\mathcal{T}(G, \pi_0)$ and $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$.*

Proof.

Part 1: σ^g is a node of $\mathcal{T}(G, \pi_0)$.

By **Lemma 1**, we have:

$$\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$$

Since $g \in \text{Aut}(G, \pi_0)$, we have $G^g = G$ and $\pi_0^g = \pi_0$, and therefore:

$$\mathcal{T}(G, \pi_0) = \mathcal{T}(G, \pi_0)^g$$

Since σ is a node of $\mathcal{T}(G, \pi_0)$, σ^g is a node of $\mathcal{T}(G, \pi_0)^g = \mathcal{T}(G, \pi_0)$.

Part 2: $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$

We apply Lemma 1 to the subtree rooted at σ . Since g maps each node of the subtree rooted at σ to a node of the subtree rooted at σ^g , and since $g \in \text{Aut}(G, \pi_0)$ implies $G^g = G$ and $\pi_0^g = \pi_0$, the same argument as in Lemma 1 gives both inclusions:

$$\mathcal{T}(G, \pi_0, \sigma)^g \subseteq \mathcal{T}(G, \pi_0, \sigma^g) \quad \text{and} \quad \mathcal{T}(G, \pi_0, \sigma^g) \subseteq \mathcal{T}(G, \pi_0, \sigma)^g$$

and therefore $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$. $\square$

Lemma 3. *Let σ be a node of (G, π_0) and let $\pi = R(G, \pi_0, \sigma)$. Then $\text{Aut}(G, \pi)$ is the **point-wise stabilizer** of σ in $\text{Aut}(G, \pi_0)$.*

Proof. We show both inclusions to establish the equality.

Part 1: $\text{Aut}(G, \pi) \subseteq \text{point-wise stabilizer of } \sigma$

Let $g \in \text{Aut}(G, \pi)$. By **R2**, for each $\sigma_i \in \sigma$, $\{\sigma_i\}$ is a singleton cell of $\pi = R(G, \pi_0, \sigma)$. Since g is an automorphism of (G, π) , it must map each cell of π to a cell of π . A singleton

$\{\sigma_i\}$ can only be mapped to itself, hence $\sigma_i^g = \sigma_i$ for every $\sigma_i \in \sigma$, so g stabilizes σ point-wise.

Part 2: point-wise stabilizer of $\sigma \subseteq \text{Aut}(G, \pi)$

Let $g \in \text{Aut}(G, \pi_0)$ be such that g stabilizes σ point-wise, that is $\sigma^g = \sigma$. We want to show that $g \in \text{Aut}(G, \pi)$. So, $\pi^g = \pi$. By **R3** applied to G , π_0 and σ :

$$R(G^g, \pi_0^g, \sigma^g) = R(G, \pi_0, \sigma)^g$$

Since $g \in \text{Aut}(G, \pi_0)$, we have $G^g = G$ and $\pi_0^g = \pi_0$, and since g stabilizes σ point-wise, $\sigma^g = \sigma$. Therefore:

$$R(G, \pi_0, \sigma) = R(G, \pi_0, \sigma)^g$$

which gives exactly $\pi = \pi^g$, and hence $g \in \text{Aut}(G, \pi)$. $\square$

Now that we have properly defined our search tree, we can discuss the node invariant function and how it allows us to compute the canonical form of the graph.

Definition 19. Let Ω be some totally ordered set. A *Node invariant* is a function

$$\phi : \mathcal{G} \times \Pi \times V^* \rightarrow \Omega$$

Such that, for any $\pi_0 \in \Pi$, $G \in \mathcal{G}$ and distinct $\sigma, \sigma' \in \mathcal{T}(G, \pi_0)$.

The function ϕ has a few properties to take into account:

1. if $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$,
then for every leaf $\sigma_1 \in \mathcal{T}(G, \pi_0, \sigma)$ and leaf $\sigma'_1 \in \mathcal{T}(G, \pi_0, \sigma')$ we have $\phi(G, \pi_0, \sigma_1) < \phi(G, \pi_0, \sigma'_1)$.
2. if $\pi = R(G, \pi_0, \sigma)$ and $\pi' = R(G, \pi_0, \sigma')$ are discrete,
then $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma') \Leftrightarrow G^\pi = G^{\pi'}$ (equality relation).
3. for any $g \in S_n$, we have $\phi(G^g, \pi_0^g, \sigma^g) = \phi(G, \pi_0, \sigma)$.
4. Leaves σ, σ' are *equivalent* if $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma')$. This is the case, if there is a unique $g \in \text{Aut}(G, \pi_0)$ such that $\sigma^g = \sigma'$, with $g = R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1}$.

Property 1 ensures that the ordering induced by ϕ is consistent across the search tree: if two sequences of equal length satisfy $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$, then this strict inequality propagates to all descendant leaves, so no leaf below σ can exceed any leaf below σ' . Property 2 states that at the leaves (where both refined partitions are discrete) the invariant captures the canonical labelling exactly: two leaves yield equal invariants if and only if they produce the same canonically labelled graph. Property 3 expresses that ϕ is invariant under permutation: applying any $g \in S_n$ to the graph, the colouring, and the vertex sequence before computing ϕ leaves the result unchanged.

Finally, Property 4 connects equivalence of leaves to the automorphism group. We recall that $R(G, \pi_0, \sigma)$ is the discrete partition obtained by applying the refinement function along the sequence σ , and can therefore be viewed as a permutation of the vertices. Since both leaves are equivalent, their node invariants are equal, which means there exists a permutation g such that:

$$R(G, \pi_0, \sigma)^g = R(G, \pi_0, \sigma')$$

Isolating g , we obtain:

$$g = R(G, \pi_0, \sigma)^{-1} \cdot R(G, \pi_0, \sigma')$$

If this permutation turns out to be an automorphism of (G, π_0) , it means that both leaves describe the same graph up to a relabelling of vertices, and are therefore equivalent.

By [Corollary 2](#), if σ is a leaf of $\mathcal{T}(G, \pi_0)$, then so is σ^g for any $g \in \text{Aut}(G, \pi_0)$. Moreover, due to the properties of ϕ , all such leaves σ^g share the same ϕ -value, and no other leaf has this value. Consequently, for each leaf σ , we can say that:

$$\text{Aut}(G, \pi_0) = \{R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1} : \sigma' \text{ is a leaf of } \mathcal{T}(G, \pi_0) \\ \text{with } \phi(G, \pi_0, \sigma') = \phi(G, \pi_0, \sigma)\}.$$

We can define the canonical form:

$$\phi^*(G, \pi_0) := \max \{\phi(G, \pi_0, \sigma) : \sigma \text{ is a leaf of } \mathcal{T}(G, \pi_0)\},$$

and let σ^* be any leaf of $\mathcal{T}(G, \pi_0)$ that achieves the maximum. Now define $C(G, \pi_0) := (G, \pi_0)^{R(G, \pi_0, \sigma^*)}$. By the properties of ϕ , $C(G, \pi_0)$ thus defined is independent of the choice of σ^* , which is consistent with the definition of [the canonical form](#).

4.2.2 Implementation strategies

Now that we have all the theoretical tools in hand, we can begin discussing the implementation of the algorithm. Our implementation is based on the definitions of the algorithm, but it is not a direct or simplistic reproduction of its source code.

Implementation Refinement function

In order to ensure that the properties of the [refinement function](#) defined in the search tree are respected, we must implement an algorithm that outputs a colouring that is equal to (or finer) than the original colouring. This motivates the need for an algorithm that computes a coarsest equitable colouring, in order to prevent refinement from being too aggressive and skipping intermediate structure in the search tree.

To construct our refinement function, we need two auxiliary functions: a function that refines a colouring so that it becomes an equitable colouring, and an individualisation function.

Definition 20. The function $F(G, \pi, \alpha)$ is our refinement algorithm (implemented in a label-invariant manner).

pseudo-code of $F(G, \pi, \alpha)$ (from [40])

```

1: Data:  $\pi$  is the input colouring and  $\alpha$  is a sequence of some cells of  $\pi$ 
2: Result: the final value of  $\pi$  is the output colouring (equitable)
3: while  $\alpha$  is not empty and  $\pi$  is not discrete do
4:   Remove some element  $W$  from  $\alpha$ 
5:   for each cell  $X$  of  $\pi$  do
6:     Let  $X_1, \dots, X_k$  be the fragments of  $X$  distinguished according to the number
       of edges from each vertex to  $W$ 
7:     Replace  $X$  by  $X_1, \dots, X_k$  in  $\pi$ 
8:     if  $X \in \alpha$  then
9:       Replace  $X$  by  $X_1, \dots, X_k$  in  $\alpha$ 
10:    else
11:      Add all but one of the largest of  $X_1, \dots, X_k$  to  $\alpha$ 
12:    end if
13:  end for
14: end while

```

This function is, at first glance, relatively straightforward to understand. However, it is important to carefully examine the different steps involved in the refinement process. We therefore describe these steps in more detail below.

As stated in the pseudo-algorithm, we are given a graph G , a colouring π , and a sequence of cells of π that will be used to refine the colouring. As long as not all cells in the sequence have been processed and the colouring is not yet discrete, we continue refining the colouring.

At each iteration, we remove a cell W from the sequence. Then, for every cell X of the current colouring, we split X into several cells $X_1, \dots, X_k$ according to the number of edges each vertex of X has with the vertices of W (with respect to the graph G). We then replace the cell X in the colouring by the newly created cells $X_1, \dots, X_k$.

If the cell X already belongs to the sequence, we replace it in the sequence by the cells $X_1, \dots, X_k$. Otherwise, we add all cells among $X_1, \dots, X_k$ to the sequence except one, namely a largest cell. If several cells have the same maximum size, one of them is chosen arbitrarily and is not added to the sequence.

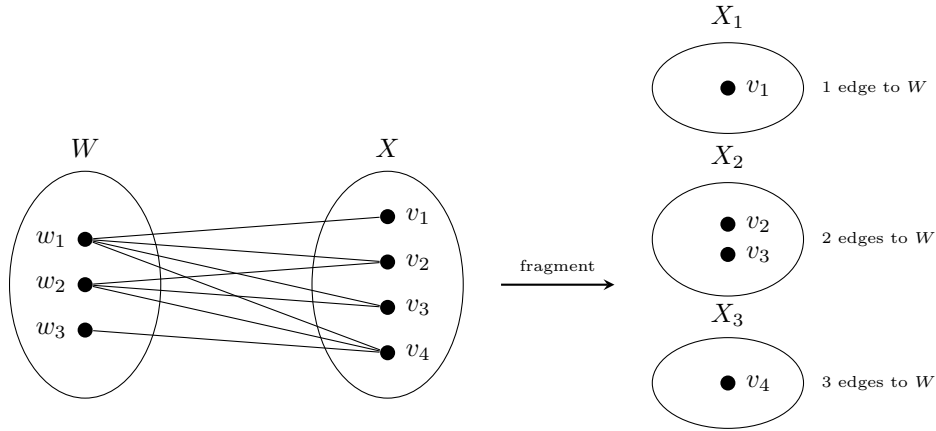


Figure 4.2: Fragmentation of cell X with respect to W

Definition 21 (Individualization). Let $\pi \in \Pi$ and let $v \in V$ be a vertex belonging to a non-singleton cell of π . The *individualization function*

$$I : \Pi \times V \rightarrow \Pi$$

maps (π, v) to the partition $\pi' = I(\pi, v)$ defined, for every $w \in V$, by

$$\pi'(w) = \begin{cases} \pi(w), & \text{if } \pi(w) < \pi(v) \text{ or } w = v, \\ \pi(w) + 1, & \text{otherwise.} \end{cases}$$

The vertex v is said to be *individualized* in π . Equivalently, $I(\pi, v)$ is obtained from π by assigning a unique colour to the vertex v .

The purpose of this function is to construct a new partition from a given partition by individualizing a vertex. More precisely, the operation assigns a unique colour to a specific vertex v by removing it from its original cell and placing it into a new singleton cell. This new cell is inserted immediately before the original cell containing v .

Now we can define our refinement function. For a sequence of vertices $v_1, v_2, \dots$, define

- $R(G, \pi_0, ()) = F(G, \pi_0, \text{a list of all the cells of } \pi_0)$
- $R(G, \pi_0, (v_1)) = F(G, I((R(G, \pi_0, ()), v_1), \{v_1\}))$
- $R(G, \pi_0, (v_1, v_2)) = F(G, I((R(G, \pi_0, (v_1)), v_2), \{v_2\}))$
- $R(G, \pi_0, (v_1, v_2, v_3)) = F(G, I((R(G, \pi_0, (v_1, v_2)), v_3), \{v_3\}))$
- and so on.

The algorithm satisfies the required properties of a refinement function and performs well in practice. However, the refinement step is the most computationally expensive part of the algorithm, which is why special care must be taken in its implementation. C'est pourquoi, pour réduire les temps de calculs, nous allons utiliser des méthodes de pruning dont nous parlerons un peu plus tard.

Implementation Target Cell Selector

For the implementation of the **target cell selector function** in the construction of the search tree, several strategies can be adopted. Choosing the right target cell at each step is important because it influences the shape and depth of the tree, and therefore the overall performance of the search.

In the original version of *Nauty*, McKay recommended selecting the first smallest non-singleton cell, as it increases the chance that the selected cell is part of an orbit of the group that stabilizes the current sequence [39]. However, later studies showed that in certain cases, it is better to simply select the first non-singleton cell, regardless of its size [33].

As a result, *Nauty* now supports two strategies for target cell selection that we can choose:

1. Always select the first smallest non-singleton cell.
2. Select the first cell that is joined in a non-trivial way to the largest number of other cells, where a non-trivial join means the number of edges between two cells is strictly between 0 and the maximum possible.

In our implementation we will only use the first strategy.

pseudo-code of targetCellSelector

- 1: **Data:** a partition $\pi = (X_1, \dots, X_k)$
- 2: **Result:** the first non-singleton cell of minimum size in π
- 3: **return** the first $X \in \pi$ such that $|X| > 1$ and $|X| = \min_{Y \in \pi, |Y| > 1} |Y|$

Intermediate Search Tree Construction

All the functions required to define the search tree have now been implemented. The following example illustrates the construction of the search tree for the graph $K_{1,3}$ using target-cell selection and equitable refinement.

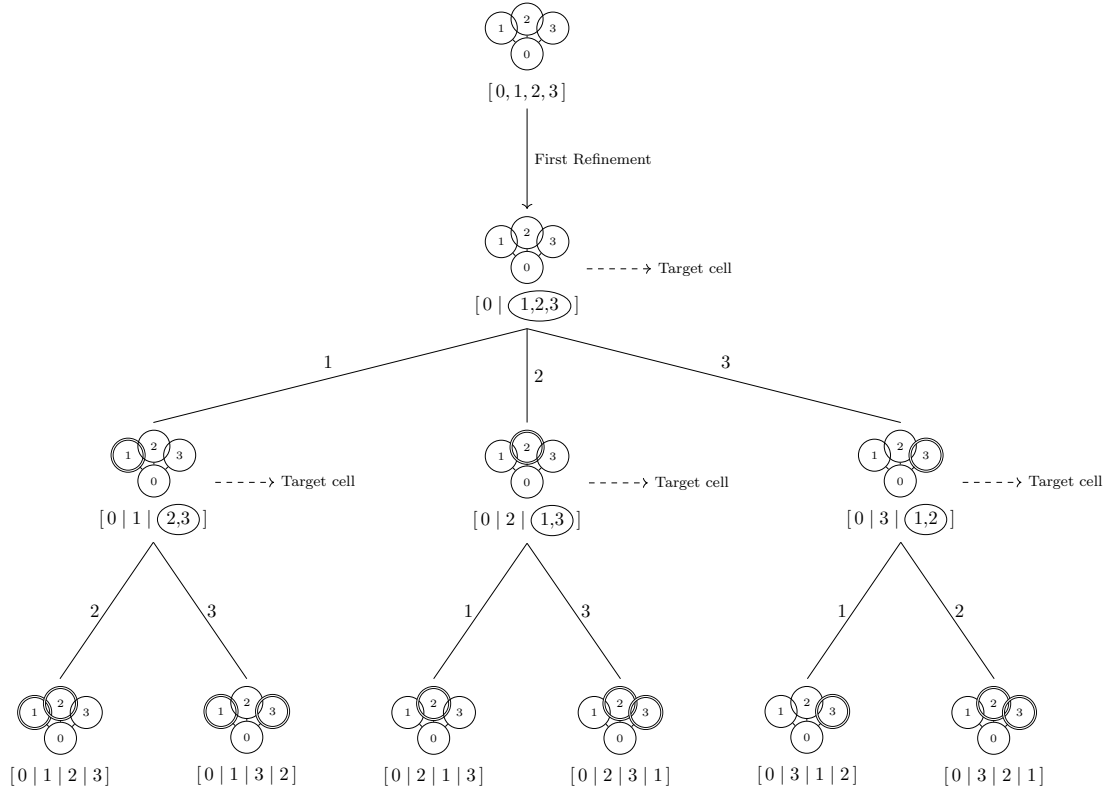


Figure 4.3: Individualized vertices are represented by double circles. The encircled set in each partition denotes the current target cell.

Implementation Node Invariant

Once the search tree construction is in place, the next step is to define the **node invariant function**, which will allow us to determine the canonical form. To compute this function, we can rely on two related sources:

1. For each node σ , there is a colouring $R(G, \pi_0, \sigma)$. We can use its properties to extract the number and size of the cells, as well as combinatorial properties of the coloured graph.
2. We can use the intermediate states of the colouring computation from the parent node (in the refinement algorithm), such as the position and size of the cells produced during the refinement procedure, and various edge counts determined during the computation. If $f(\sigma)$ is a function of this information—computed during the calculation of $R(G, \pi_0, \sigma)$ and from the resulting coloured graph—then the vector $(f([\sigma]_0), f([\sigma]_1), \dots, f(\sigma))$, with lexicographic ordering, satisfies the characteristics required for a node invariant. In *Nauty*, the second source is used: the value of $f(\sigma)$ is an integer, and the pruning rules are applied as each node is computed.

Another potential method is based on the concept of the *quotient graph* $Q(G, \pi)$ (see [Definition 6](#)), assuming that the colouring π of graph G is equitable. All vertices of $Q(G, \pi)$ are cells of π , and they include information about the size and number of the cell. For any two cells $C_1, C_2 \in \pi$, possibly equal, the corresponding vertices in $Q(G, \pi)$ are connected by an edge labelled with the number of edges in G between C_1 and C_2 . The node invariant already computed by *Nauty* is a deterministic function of the sequence of quotient graphs $Q(G, R(G, \pi_0, [v]_i))$ for $i = 0, \dots, |v|$. Therefore, it would be possible to use the sequence of quotient graphs to obtain information about the cells, but this would be far too costly in both time and memory.

In our implementation, we will use the first source described above. We rely on the number and size of the cells, and we apply combinatorial properties of the coloured graph to compute our node invariant. Several invariant methods have been implemented in order to test their efficiency and computational speed.

Since this function is called at each node creation, it is important that its computation remains fast in order to avoid slowing down the algorithm. The following sections detail each of these methods, along with a brief discussion of their computational complexity and how they compare to one another.

Here is the exhaustive list of invariant methods that we have implemented so far:

1. *Number of triangles per cell*: for each cell of the partition, we compute the number of triangles contained in that cell. We then obtain a vector where each entry corresponds to the number of triangles in the corresponding cell. Finally, we apply a lexicographic ordering to this vector to compute our node invariant.

For counting the number of triangles in a cell, the most classical approach enumerates all triplets of vertices (u, v, w) from the cell and checks for each one whether all three edges exist, running in $O(n^3)$ where n is the number of vertices in the cell. A faster approach exploits the sorted neighbor lists $N(v)$: for each vertex $u \in C$, one iterates over its neighbors v in the cell with $v > u$, then looks for common neighbors w of both u and v still in the cell with $w > v$. Since the neighbor lists are sorted, each such lookup runs in $O(d)$ where d is the largest number of neighbors that any vertex in the cell has, yielding an overall complexity of $O(n \cdot d^2)$.

Pseudo-code: Number of triangles per cell

```

1: Data: A graph  $G$ , a cell  $C = \{v_1, \dots, v_k\}$ 
2:    $N(u)$  denotes the ordered neighbor list of  $u$  in  $G$ 
3: Result: The number of triangles contained in  $C$ 
4:  $count \leftarrow 0$ 
5: for each  $u \in C$  do
6:   for each  $v \in N(u) \cap C$  such that  $v > u$  do
7:     for each  $w \in N(u) \cap N(v) \cap C$  such that  $w > v$  do
8:        $count \leftarrow count + 1$ 
9:     end for
10:   end for
11: end for
12: return  $count$ 

```

[Rodolphe: les prochaines méthodes d'invariants arrivent mais la structure est là]

4.2.3 Pruning strategies

To avoid generating unnecessary information in our search tree, which would result in an overly large tree, we aim to construct the most optimized tree possible. To achieve this, we can apply *pruning* techniques to the tree: eliminating redundant branches so that only a fraction of the original tree needs to be generated. We can define two types of pruning strategies to optimize our construction:

$P_A(\sigma, \sigma')$: Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) > \phi(G, \pi_0, \sigma')$. Then, we will remove $T(G, \pi_0, \sigma')$.

$P_B(\sigma, \sigma')$: Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) \neq \phi(G, \pi_0, \sigma')$. Then, we will remove $T(G, \pi_0, \sigma')$.

$P_C(\sigma, g)$: Suppose $g \in \text{Aut}(G, \pi_0)$ and suppose $\sigma < \sigma'$ are nodes of $T(G, \pi_0)$ such that $\sigma^g = \sigma'$. Then, we will remove $T(G, \pi_0, \sigma')$.

Despite these pruning strategies, we must ensure that our search tree, along with the node invariant function, remains functionally correct. To this end, we require a theorem that guarantees the validity of the overall construction.

Theorem 4. *Consider any $G \in \mathcal{G}$ and $\pi_0 \in \Pi$*

- *Suppose any sequence of operations of the form $P_A(\sigma, \sigma')$ or $P_C(\sigma, g)$ are performed. Then there remains at least one leaf σ_1 with $\phi(G, \pi_0, \sigma_1) = \phi^*(G, \pi_0)$.*
- *Let σ_0 be some fixed leaf of $T(G, \pi_0)$. Suppose any sequence of operations of the form $P_B(\sigma, \sigma')$ $P_C(\sigma'', g)$ are performed, where $\phi(G, \pi_0, \sigma'') \neq \phi(G, \pi_0, [\sigma_0]_{|\sigma''|})$. Let $g_1, \dots, g_k$ be the automorphisms used in the operations P_C that were performed, and let $A = \{g \in \text{Aut}(G, \pi_0) : v_0^g \text{ is a remaining leaf}\}$. Then $\text{Aut}(G, \pi_0)$ is generated by $\{g_1, \dots, g_k\} \cup A$.*

Proof. To prove the first point, let v_1 denote the lexicographically smallest leaf satisfying $\phi(G, \pi_0, v_1) = \phi^*(G, \pi_0)$. Operation P_A only removes branches whose invariant cannot reach the maximal value, while operation P_C always preserves the lexicographically smallest representative of an automorphism class. Consequently, v_1 cannot be removed.

Therefore, at least one leaf with maximal invariant always remains after any sequence of pruning operations.

For the second point, let $g \in \text{Aut}(G, \pi_0)$ and consider the corresponding leaf v_0^g .

If v_0^g is still present after pruning, then by definition $g \in A$.

Otherwise, v_0^g must have been removed by an operation $P_C(v'', g_i)$. It cannot be removed by $P_B(v', v)$ because it only removes leaves inequivalent to v_0 . This implies that there exists a lexicographically smaller equivalent leaf, namely

$$v_0^{gg_i^{-1}} < v_0^g.$$

If this new leaf has also been removed, the same reasoning applies. Repeating this process produces a strictly decreasing sequence of equivalent leaves

$$v_0^g > v_0^{gg_i^{-1}} > v_0^{gg_i^{-1}g_j^{-1}} > \dots$$

Since the lexicographic order on the finite set of leaves is well-founded, this sequence must eventually terminate at a remaining leaf. Hence, there exists $h \in \langle g_1, \dots, g_k \rangle$ such that v_0^{gh} is a remaining leaf. By definition of A , we have $gh \in A$, and therefore $g = (gh)h^{-1}$ where $gh \in A$ and $h \in \langle g_1, \dots, g_k \rangle$.

Since g was arbitrary, every automorphism belongs to the group generated by

$$\{g_1, \dots, g_k\} \cup A,$$

which proves the result. □

The P_A pruning strategy is relatively straightforward to implement. At each level of the search tree, we simply compare the node invariant of the node currently being explored with the best node invariant encountered so far at the same level. If the invariant of the current node is smaller than the best one found previously, then the corresponding subtree can safely be discarded.

The P_B strategy is slightly more involved. Instead of comparing the current node with the best node encountered so far, we compare its node invariant with that of a reference node. This reference node is chosen as the node at the same level lying on the first path leading to the first leaf. If the two node invariants differ, the current subtree is pruned. Conversely, branches whose node invariants coincide with those of the reference path remain candidates for discovering new automorphisms.

Finally, the P_C strategy exploits the automorphisms that have already been discovered during the search. Suppose that an automorphism $g \in \text{Aut}(G, \pi_0)$ maps the current node σ to another node σ^g that has already been explored. Since the corresponding subtrees are isomorphic, exploring both of them would be redundant. Consequently, the subtree rooted at σ^g can be discarded.

Implementing this strategy is considerably more challenging than implementing P_A or P_B , and several approaches have been proposed over the years. In the earliest versions of *Nauty*, pruning relied solely on the automorphisms discovered during the search. More recent versions additionally provide an optional *Random Schreier* method, which *Nauty* adopted in place of a deterministic alternative for its greater efficiency, and which constructs a

randomized approximation of the automorphism group from the automorphisms already found. This often enables more aggressive pruning, although its effectiveness depends on the input graph; for some graph families it provides little or no improvement and may even introduce additional overhead.

In this work, we implement the original approach, which relies only on the automorphisms discovered during the search. To represent and manipulate the automorphism group efficiently, we use the *Schreier-Sims algorithm*, which constructs a base and strong generating set (BSGS) together with the associated stabilizer chain, allowing efficient orbit computations and pruning decisions.

Schreier-Sims Algorithm

A permutation group $G \leq S_n$ given by a set of generators can be extremely large, growing combinatorially with n , so working with it by listing its elements one by one is simply not an option. The Schreier-Sims algorithm avoids this by building a compact representation of G , from which the order $|G|$, membership testing, and orbit computations can all be obtained efficiently.

The idea is to shrink the group step by step. Starting from G , we choose a point b_1 and restrict our attention to the permutations of G that happen to fix it: this smaller group is the *stabilizer* of b_1 . We then choose a second point b_2 and repeat the same reduction inside this smaller group, and so on, until only the identity permutation is left. The sequence of chosen points $b_1, \dots, b_k$ is called a *base*.

This reduction is made computationally cheap by two related notions. The *orbit* of the current base point is the set of points it can be mapped to by the current group; it can be computed by a breadth-first search using only the group's generators. This same search yields a *transversal*, that is, for every point in the orbit, a permutation of the current group mapping the base point to it. Combining the transversal with the current generators, through the so-called *Schreier generators*, produces a generating set for the next, smaller stabilizer, without ever enumerating the elements of the group itself.

This process gives a base together with a generating set for each successive stabilizer. It is called a *Base and Strong Generating Set* (BSGS). It also gives an immediate way to compute the order of G : by the orbit-stabilizer theorem, $|G|$ is simply the product of the orbit sizes obtained at each step,

$$|G| = \prod_{i=1}^k o_i,$$

where o_i is the size of the orbit of b_i computed at step i .

To better understand the construction, we consider the graph $K_{1,3}$. We build the BSGS associated with $\text{Aut}(K_{1,3})$ using the base $(b_1 = 1, b_2 = 2)$.

We observe that each edge at level i is labelled by an element $\sigma_p^{(i)}$ of the transversal associated with the orbit of b_{i+1} under $G^{(i)}$. Each leaf corresponds to an element $g \in G$, obtained by composing the transversal representatives along the path from the root. The $3 \times 2 \times 1 = 6$ leaves therefore represent exactly the six elements of $\text{Aut}(K_{1,3})$.

Schreier-Sims for *Nauty*

Let G be the input graph and π_0 its initial partition.

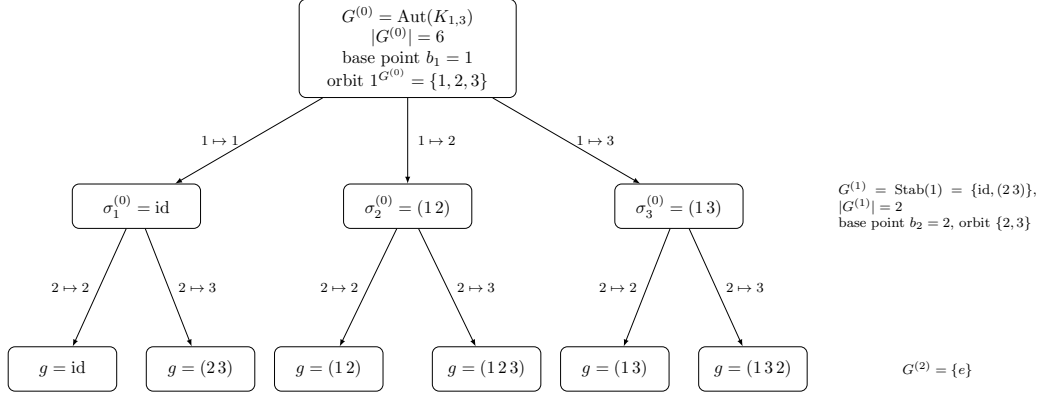


Figure 4.4: Tree associated with the BSGS of $\text{Aut}(K_{1,3})$ with base $(b_1 = 1, b_2 = 2)$.

Nauty performs a depth-first traversal of the search tree, and at each leaf it is possible to discover a new automorphism of the graph and add it to the currently known automorphism group. Whenever two distinct leaves σ_0 (the first leaf) and σ' produce the same invariant, the permutation $g = R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1}$ is an automorphism of G .

These automorphisms, discovered one by one during the depth-first traversal, form the generators from which the group structure is built. It is therefore not necessary to know $\text{Aut}(G, \pi_0)$ in advance: the generators emerge dynamically during the search, and the group structure is updated incrementally as new automorphisms are discovered.

The automorphism g is then sifted through the existing stabilizer chain, correcting its action on each base point successively. It is either absorbed by an existing level, in which case it provides no new information, or it is identified as a new strong generator and enriches the chain.

Each automorphism discovered at a leaf is propagated to the ancestor nodes for which it remains relevant. The accumulated group information is then used when selecting the next vertex to individualize. Once a target cell has been chosen, its vertices are partitioned into orbits under the currently known automorphism group. Two vertices belonging to the same orbit necessarily lead to isomorphic subtrees, since they are related by an automorphism already known to the algorithm. Consequently, it is sufficient to explore a single representative of each orbit, while all other vertices can be pruned without exploration.

This is a direct application of the pruning operation P_C .

To get a concrete idea of what this looks like in code, here is a simplified pseudo-code of the Schreier-Sims method for *Nauty*.

Pseudo-code of Schreier-Sims method for Nauty

```

1: Data: graph  $G$ , initial partition  $\pi_0$ , reference leaf  $\sigma_0$ , current base  $B$ 
2: Result: BSGS
3: for each leaf  $\sigma'$  reached by the depth-first search do
4:   if  $\phi(G, \pi_0, \sigma') = \phi(G, \pi_0, \sigma_0)$  then
5:      $g \leftarrow R(G, \pi_0, \sigma') R(G, \pi_0, \sigma_0)^{-1}$  ▷ candidate automorphism
6:     if  $g \neq \text{id}$  then
7:       INSERTGENERATOR( $g$ , level 1)
8:     end if
9:   end if
10: end for

11: procedure INSERTGENERATOR( $g$ , level  $i$ )
12:   if  $B$  has no point at level  $i$  then
13:     choose a new base point  $b_i$  moved by  $g$ 
14:   end if
15:   add  $g$  to the strong generators at level  $i$ 
16:   GROWORBIT( $i$ ) ▷ updates the orbit and transversal at level  $i$ 
17:   CLOSESTABILIZER( $i$ ) ▷ Schreier generators, recurses if needed
18: end procedure

```

Random Schreier method

The main difference between the classical Schreier-Sims algorithm seen previously and the Random Schreier method lies in the way the BSGS is constructed. Instead of deterministically maintaining an exact base and strong generating set from every newly discovered automorphism, the Random Schreier method adopts a probabilistic approach. It repeatedly generates random products of the automorphisms already known and inserts these permutations into the Schreier-Sims structure.

The objective is not to compute the complete BSGS as early as possible, but rather to obtain a sufficiently rich approximation of the subgroup generated by the discovered automorphisms. This approximation often provides accurate orbit information much earlier during the search, allowing the P_C pruning strategy to eliminate redundant branches sooner. Consequently, the search tree may become significantly smaller.

Since the method relies on randomization, its effectiveness depends on the input graph. For many highly symmetric graphs, it substantially improves the pruning process. However, for graphs with few automorphisms, the additional computations required to generate and process random group elements may outweigh the benefits. For this reason, the Random Schreier method is implemented as an optional feature in *Nauty*.

[Rodolphe: Mettre complexité]

[Rodolphe: Mettre graphe final de pruning exemple]

4.2.4 Final Search Tree Construction

We can put in practice our different pruning strategies on our search tree. *Nauty* will generate its tree in depth-first order. The lexicographically least leaf σ_1 is kept and if the canonical labelling is sought (rather than just the automorphism group), the leaf σ^* with

the greatest invariant discovered so far is also kept. A non-discrete node σ is pruned if neither $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, [\sigma_1]_{|\sigma|})$ or $\phi(G, \pi_0, \sigma) \geq \phi(G, \pi_0, [\sigma^*]_{|\sigma|})$. Such operations have both type $P_A(\sigma^*, \sigma)$ and $P_B(\sigma_1^*, \sigma)$. We can therefore conclude that only a subset of the tree will be generated, and that the generated leaves are those that may be “better” than the leaves already discovered.

Pseudo-code of canonical labelling

```

1: Data: A graph  $G = (V, E)$ 
2: Result: The canonical form of  $G$ 
3:  $\pi \leftarrow (\{0, 1, \dots, |V| - 1\})$ 
4:  $\pi \leftarrow F(G, \pi, \text{a list of all the cell of } \pi)$ 
5:  $\pi^* \leftarrow \pi$  ▷  $\pi^*$  the best partition
6:  $\pi_1 \leftarrow ()$  ▷  $\pi_1$  the partition of first leaf
7:  $\sigma \leftarrow (), \quad \sigma^* \leftarrow ()$  ▷ current and best vertex sequences
8: procedure NAUTY( $G, \pi$ )
9:   if  $\pi$  is discrete then
10:     if  $\pi^*$  is not discrete then ▷ no leaf found yet
11:        $\pi^* \leftarrow \pi, \sigma^* \leftarrow \sigma, \pi_1 \leftarrow \pi$ 
12:       BSGS  $\leftarrow$  Base ( $\sigma$ ) ▷ initialize Base
13:     else if  $\phi(G, \pi, \sigma) > \phi(G, \pi^*, \sigma^*)$  then
14:        $\pi^* \leftarrow \pi, \sigma^* \leftarrow \sigma$  ▷ better leaf found
15:     end if
16:     return
17:   end if
18:   if  $\pi^*$  is discrete and  $\phi(G, \pi, \sigma) < \phi(G, \pi^*, \sigma^*)$  and  $\phi(G, \pi, \sigma) \neq \phi(G, \pi_1, \sigma_1)$  then
19:     return ▷ Pruning  $P_A + P_B$ : cannot beat  $\pi^*$ , nor tie  $\pi_1$ 
20:   end if
21:   Cell  $\leftarrow T(G, \pi, \sigma)$ 
22:    $R \leftarrow$  orbit representatives of Cell ▷ Pruning  $P_C$ 
23:   for each  $\nu \in R$  do
24:      $\pi_\nu \leftarrow I(\pi, \nu)$ 
25:      $\pi_\nu \leftarrow F(G, \pi_\nu, \{\nu\})$ 
26:     Append  $\nu$  to  $\sigma$ 
27:     NAUTY( $G, \pi_\nu$ )
28:     Remove last entry from  $\sigma$ 
29:   end for
30: end procedure
31:  $g \leftarrow$  canonical vertex ordering induced by  $\pi^*$ 
32: return  $G^g$  ▷ Apply  $g$  to  $G$  to get the canonical form

```

The graph encoding is not fixed; however, we adopt the same approach as nauty, namely representing input graphs using the graph6 format [41]. This choice enables more efficient testing of our algorithm.

[Rodolphe: Tu crois qu'il y a un problème de convention ici et que je devrais juste mettre des π_0 partout?]

Bibliography

- [1] A. V. Aho and J. E. Hopcroft. *The design and analysis of computer algorithms*. Pearson Education India, 1974.
- [2] V. L. Arlazarov, I. I. Zuev, A. V. Uskov, and I. Faradzhev. An algorithm for the reduction of finite non-oriented graphs to canonical form. *USSR Computational Mathematics and Mathematical Physics*, 14(3), 1974.
- [3] L. Babai. Graph isomorphism in quasipolynomial time. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, pages 684–697, 2016.
- [4] L. Babai, P. Erdos, and S. M. Selkow. Random graph isomorphism. *SIAM Journal on computing*, 9(3):628–635, 1980.
- [5] L. Babai, W. M. Kantor, and E. M. Luks. Computational complexity and the classification of finite simple groups. In *24th Annual Symposium on Foundations of Computer Science (Sfcs 1983)*, pages 162–171. IEEE, 1983.
- [6] S. Baruah. Feasibility analysis for hpc-dag tasks. *Real-Time Systems*, 58:1–19, June 2022. DOI: [10.1007/s11241-022-09380-z](https://doi.org/10.1007/s11241-022-09380-z).
- [7] C. Berkholtz, P. Bonsma, and M. Grohe. Tight lower and upper bounds for the complexity of canonical colour refinement. *Theory of Computing Systems*, 60:581–614, 2017.
- [8] H. L. Bodlaender. Polynomial algorithms for graph isomorphism and chromatic index on partial k -trees. *Journal of Algorithms*, 11(4):631–643, 1990.
- [9] G. Brinkmann, J. Goedgebeur, and N. Van Cleemput. The history of the generation of cubic graphs. *International Journal of Chemical Modeling*, 5(2-3):203–225, 2013.
- [10] A. Cardon and M. Crochemore. Partitioning a graph in $\mathcal{O}(|A| \log_2 |V|)$. *Theoretical Computer Science*, 19(1):85–98, 1982.
- [11] C. J. Colbourn and K. S. Booth. Linear time automorphism algorithms for trees, interval graphs, and planar graphs. *SIAM Journal on Computing*, 10(1):203–225, 1981.
- [12] D. G. Corneil and C. C. Gotlieb. An efficient algorithm for graph isomorphism. *Journal of the ACM (JACM)*, 17(1):51–64, 1970.
- [13] P. T. Darga, M. H. Liffiton, K. A. Sakallah, and I. L. Markov. Exploiting structure in symmetry detection for CNF. In *Proceedings of the 41st Annual Design Automation Conference*, pages 530–534, 2004.
- [14] B. Das, M. K. Enduri, and I. V. Reddy. Polynomial-time algorithm for isomorphism of graphs with clique-width at most three. *Theoretical Computer Science*, 819:9–23, 2020.

- [15] S. Datta, N. Limaye, P. Nimbhorkar, T. Thierauf, and F. Wagner. Planar graph isomorphism is in log-space. In *2009 24th Annual IEEE Conference on Computational Complexity*, pages 203–214. IEEE, 2009.
- [16] R. Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, Heidelberg, 6th edition, 2025. ISBN: 978-3-662-53621-6.
- [17] P. Dobsan. Pynauty: a python interface for nauty, 2023. URL: <https://github.com/pdobsan/pynauty>. Accessed: 2025-05-11.
- [18] T. Doe and contributors. Nautygraphs.jl: a julia interface for nauty, 2023. URL: <https://discourse.julialang.org/t/new-nautygraphs-jl/119029>. Accessed: 2025-05-11.
- [19] I. S. Filotti and J. N. Mayer. A polynomial-time algorithm for determining the isomorphism of graphs of fixed genus. In *Proceedings of the twelfth annual ACM symposium on Theory of computing*, pages 236–243, 1980.
- [20] P. Foggia, C. Sansone, M. Vento, et al. A performance comparison of five algorithms for graph isomorphism. In *Proceedings of the 3rd IAPR TC-15 Workshop on Graph-based Representations in Pattern Recognition*, pages 188–199, 2001.
- [21] M. Frank and M. Codish. Logic programming with graph automorphism: integrating nauty with prolog (tool description). *Theory and Practice of Logic Programming*, 16(5-6):688–702, 2016.
- [22] M. R. Garey and D. S. Johnson. *Computers and intractability*, volume 29. wh freeman New York, 2002.
- [23] O. Goldreich, S. Micali, and A. Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM (JACM)*, 38(3):690–728, 1991.
- [24] M. Grohe. Structural and logical approaches to the graph isomorphism problem. In *SODA*, page 188, 2012.
- [25] M. Grohe, K. Kersting, M. Mladenov, and E. Selman. Dimension reduction via colour refinement. In *Algorithms-ESA 2014: 22th Annual European Symposium, Wroclaw, Poland, September 8-10, 2014. Proceedings 21*, pages 505–516. Springer, 2014.
- [26] M. Grohe and P. Schweitzer. The graph isomorphism problem. *Communications of the ACM*, 63(11):128–134, 2020.
- [27] J. E. Hopcroft and J. K. Wong. Linear time algorithm for isomorphism of planar graphs (preliminary report). In *Proceedings of the Sixth Annual ACM Symposium on Theory of Computing*, STOC '74, pages 172–184, Seattle, Washington, USA. Association for Computing Machinery, 1974. ISBN: 9781450374231. DOI: [10.1145/800119.803896](https://doi.org/10.1145/800119.803896). URL: <https://doi.org/10.1145/800119.803896>.
- [28] J. E. Hopcroft and J.-K. Wong. Linear time algorithm for isomorphism of planar graphs (preliminary report). In *Proceedings of the sixth annual ACM symposium on Theory of computing*, pages 172–184, 1974.
- [29] A. Ivrii and O. Strichman. Exploiting isomorphic subgraphs in sat. In *2021 Formal Methods in Computer Aided Design (FMCAD)*, pages 204–211, 2021. DOI: [10.34727/2021/isbn.978-3-85448-046-4_29](https://doi.org/10.34727/2021/isbn.978-3-85448-046-4_29).
- [30] T. Junttila and P. Kaski. Conflict propagation and component recursion for canonical labeling. In *International Conference on Theory and Practice of Algorithms in (Computer) Systems*, pages 151–162. Springer, 2011.

- [31] S. Kiefer. *Power and limits of the Weisfeiler-Leman algorithm*. PhD thesis, Dissertation, RWTH Aachen University, 2020, 2020.
- [32] S. Kiefer, I. Ponomarenko, and P. Schweitzer. The weisfeiler–leman dimension of planar graphs is at most 3. *Journal of the ACM (JACM)*, 66(6):1–31, 2019.
- [33] A. Kirk. Efficiency considerations in the canonical labelling of graphs. Technical report, Technical report TR-CS-85-05, Computer Science Department, Australian, 1985.
- [34] J. Köbler, S. Kuhnert, B. Laubner, and O. Verbitsky. Interval graphs: canonical representations in logspace. *SIAM Journal on Computing*, 40(5):1292–1315, 2011.
- [35] J. L. López-Presa, A. F. Anta, and L. N. Chiroque. Conauto-2.0: fast isomorphism testing and automorphism group computation. *arXiv:1108.1060*, 2011.
- [36] E. M. Luks. Isomorphism of graphs of bounded valence can be tested in polynomial time. *Journal of computer and system sciences*, 25(1):42–65, 1982.
- [37] R. Mathon. A note on the graph isomorphism counting problem. *Information Processing Letters*, 8(3):131–136, 1979.
- [38] B. McKay. Isomorph-free exhaustive generation. *Journal of Algorithms*, 26(2):306–324, 1998.
- [39] B. D. McKay. Practical graph isomorphism, 1981.
- [40] B. D. McKay and A. Piperno. Practical graph isomorphism, II. *Journal of symbolic computation*, 60:94–112, 2014.
- [41] B. D. McKay. Description of graph6, sparse6 and digraph6 encodings. <https://users.cecs.anu.edu.au/~bdm/data/formats.txt>, 2022.
- [42] B. D. McKay. *nauty and Traces User’s Guide (Version 2.8.9)*. <https://users.cecs.anu.edu.au/~bdm/nauty/nug28.pdf>. Australian National University. Canberra, Australia, Aug. 2024.
- [43] H. L. Morgan. The generation of a unique machine description for chemical structures a technique developed at chemical abstracts service. *Journal of chemical documentation*, 5(2):107–113, 1965.
- [44] W. Myrvold and W. Kocay. Errors in graph embedding algorithms. *Journal of Computer and System Sciences*, 77(2):430–438, 2011.
- [45] D. Neuen and P. Schweitzer. An exponential lower bound for individualization-refinement algorithms for graph isomorphism. In *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2018, pages 138–150, Los Angeles, CA, USA. Association for Computing Machinery, 2018.
- [46] R. Parris and R. Read. A coding procedure for graphs. *Scientific Report. UWI/CC*, 10, 1969.
- [47] A. Piperno. Search space contraction in canonical labeling of graphs. *arXiv preprint arXiv:0804.4881*, 2008.
- [48] I. N. Ponomarenko. Isomorphism problem for classes of graphs closed under contractions. *Zapiski Nauchnykh Seminarov POMI*, 174:147–177, 1988.
- [49] L. C. Ray and R. A. Kirsch. Finding chemical records by digital computers. *Science*, 126(3278):814–819, 1957.
- [50] R. C. Read and D. G. Corneil. The graph isomorphism disease. *Journal of graph theory*, 1(4):339–363, 1977.

- [51] D. C. Schmidt and L. E. Druffel. A fast backtracking algorithm to test directed graphs for isomorphism using distance matrices. *Journal of the ACM (JACM)*, 23(3):433–445, 1976.
- [52] U. Schöning. Graph isomorphism is in the low hierarchy. *Journal of Computer and System Sciences*, 37(3):312–323, 1988.
- [53] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(9), 2011.
- [54] M. Sipser. Introduction to the theory of computation. *ACM Sigact News*, 27(1):27–29, 1996.
- [55] E. H. Sussenguth. A graph-theoretic algorithm for matching chemical structures. *Journal of Chemical Documentation*, 5(1):36–43, 1965.
- [56] J. Torán. On the hardness of graph isomorphism. *SIAM Journal on Computing*, 33(5):1093–1108, 2004.
- [57] S. H. Unger. Git—a heuristic program for testing pairs of directed line graphs for isomorphism. *Communications of the ACM*, 7(1):26–34, 1964.